

# Excel Knowledge Extractor — Agent Instructions

## 1. Project Purpose

This repository converts complex Japanese Excel workbooks into detailed English business knowledge resources.

The source Excel files may contain:

- Japanese text
- Multiple worksheets
- Irregular table structures
- Multiple tables on one sheet
- Merged cells
- Multi-row and multi-column headers
- Hidden rows and columns
- Formulas
- Cross-sheet references
- Named ranges
- Conditional formatting
- Comments and notes
- Images positioned over cells
- Screenshots
- Process diagrams
- Flowcharts
- Charts
- Shapes
- Irregular visual layouts

The objective is NOT simply to translate cell text.

The objective is to reconstruct the business meaning of the workbook and produce trustworthy English documentation that can later be consumed by an LLM/RAG system.

---

## 2. Core Pipeline

The implementation must follow this pipeline:

Excel → Ingestion → Structural extraction → Visual/image extraction → Intermediate Representation (IR) → Semantic analysis → Japanese-to-English translation → Business rule extraction → Formula interpretation → Relationship detection → English documentation → Semantic chunking → Embeddings → Optional vector database → RAG

Do not bypass the Intermediate Representation layer.

---

### 3. Core Design Principle

Never directly convert:

Excel → Markdown

Instead:

Excel → Workbook IR → Semantic IR → Documentation

The Intermediate Representation is the canonical representation of the workbook.

All downstream components should operate primarily on the IR rather than directly on openpyxl objects.

---

### 4. Source Traceability

Every extracted piece of knowledge must preserve its original location.

Knowledge objects should contain source metadata wherever applicable:

- workbook filename
- worksheet name
- cell/range
- row number
- column number
- image ID
- image anchor
- formula
- section
- table
- source type

Example:

```
{ "source": { "workbook": "sales.xlsx", "sheet": "営業実績", "range": "H12:H25" } }
```

Never generate business knowledge without retaining source information.

---

### 5. Workbook Ingestion

Use Python as the primary implementation language.

Preferred libraries:

- openpyxl for workbook/cell/formula/style extraction
- Pillow for image handling
- zipfile/lxml when low-level XLSX XML access is required
- LibreOffice headless for rendering worksheets to PDF/image when visual reconstruction is necessary

Do not assume that openpyxl alone contains all information required to understand a visually complex workbook.

The XLSX file is a ZIP package containing XML and media resources. Low-level XML inspection may be required for drawing anchors and image relationships.

---

## 6. Workbook Parsing Requirements

Extract at minimum:

- workbook name
- workbook properties
- sheet names
- sheet order
- dimensions
- used ranges
- cell values
- formulas
- formula cells
- merged ranges
- hidden rows
- hidden columns
- row heights
- column widths
- styles where useful
- number formats
- comments
- hyperlinks
- named ranges
- data validation where relevant
- conditional formatting where relevant
- sheet visibility

Never discard formulas.

For formula cells store both:

- original formula
- interpreted business meaning

Example:

Formula:

=IFERROR(H12/G12,0)

Business interpretation:

Achievement Rate = Actual Sales / Budget. If the denominator is unavailable or zero, return 0.

---

## 7. Image Handling

Images are first-class knowledge objects.

Do not assume an image belongs to one cell.

Excel images may be anchored over a range of cells.

For each image attempt to determine:

- image ID
- worksheet
- original media file
- image format
- width
- height
- anchor position
- starting cell
- ending cell
- related cell region

Example:

```
{ "image_id": "img_001", "sheet": "業務フロー", "anchor": { "from": "B5", "to": "J24" } }
```

Images must be extracted into:

output/images/

---

## 8. Image Analysis

For every meaningful image, create:

1. OCR text
2. Japanese text
3. English translation
4. visual description
5. semantic/business interpretation

#### 6. source location

Do not rely only on OCR.

For diagrams and flowcharts, preserve relationships such as:

$A \rightarrow B \rightarrow C$

rather than producing only:

A B C

The semantic representation must preserve sequence, hierarchy, dependencies, and relationships where detectable.

If a vision model is unavailable, gracefully fall back to:

- OCR
- image metadata
- source-cell context

Do not invent visual information.

---

## 9. Visual Sheet Reconstruction

Complex Excel workbooks often encode meaning using:

- merged cells
- borders
- colors
- indentation
- spacing
- alignment
- images
- arrows
- shapes

Therefore the system should support rendering a worksheet to an image/PDF using LibreOffice when needed.

The visual representation should complement, not replace, the logical extraction.

Use both:

Logical Excel representation

+ Rendered visual representation

---

## 10. Intermediate Representation

Create strongly typed Python models.

At minimum:

WorkbookIR SheetIR SectionIR TableIR CellIR ImageIR FormulaIR BusinessRuleIR RelationshipIR KnowledgeChunkIR

Use Pydantic models where practical.

The IR must be serializable to JSON.

Example:

```
{ "workbook": "sales.xlsx", "sheets": [ { "name_ja": "営業実績", "name_en": "Sales Performance",  
"sections": [], "tables": [], "images": [], "formulas": [], "business_rules": [] } ] }
```

---

## 11. Structure Detection

The system must attempt to identify:

- title regions
- section headers
- table headers
- multi-level headers
- row groups
- column groups
- tables
- subtotals
- totals
- repeated blocks
- notes
- legends
- master-data regions
- calculation regions

Do not assume that the first row is the header.

Do not assume every non-empty rectangular area is a table.

---

## 12. Japanese-to-English Translation

Translation must be contextual.

Never translate each cell independently when surrounding context is available.

For ambiguous Japanese terms, use:

- sheet name
- section
- neighboring headers
- formulas
- related cells
- glossary
- business context

Maintain a Japanese-English glossary.

Example:

売上 → Actual Sales

予算 → Budget

実績 → Actual Result

達成率 → Achievement Rate

The glossary should be context-aware rather than purely dictionary-based.

---

## 13. Semantic Analysis

The semantic analysis stage must identify:

- business concepts
- metrics
- dimensions
- entities
- business rules
- calculations
- assumptions
- dependencies
- workflows
- relationships
- source systems where identifiable
- outputs/reports

Example:

Concept:

Achievement Rate

Definition:

Measures actual performance against the applicable target.

Formula:

$\text{Actual Sales} / \text{Budget} \times 100$

Interpretation:

100% means the target was exceeded.

---

## 14. Formula Interpretation

Every important formula should have:

- original Excel formula
- source cells
- referenced sheets
- referenced ranges
- formula category
- English explanation
- business interpretation

Do not replace the original formula with the explanation.

Keep both.

---

## 15. Cross-Sheet Relationships

Detect relationships such as:

Sales Sheet → references Product Master

Sales Sheet → references Department Master

Sales Summary → aggregates Sales Sheet

Dashboard → consumes Sales Summary

Represent relationships explicitly.

Example:

```
{ "source": "Sales Performance", "target": "Product Master", "relationship": "references" }
```

---

## 16. Documentation Generation

Generate human-readable English Markdown.

Recommended output:

```
output/documentation/ ├── workbook_overview.md ├── sheets/ | | ├── sales_performance.md
| | ├── product_master.md | | ├── approval_flow.md ├── business_rules.md ├──
formula_dictionary.md ├── glossary_ja_en.md ├── cross_sheet_relationships.md
```

Documentation must be understandable by a new English-speaking employee who has never seen the original Japanese workbook.

---

## 17. Documentation Quality

Documentation should answer:

- What is this sheet?
- Why does it exist?
- Who uses it?
- What does each section mean?
- What does each important column mean?
- How are important metrics calculated?
- What business rules exist?
- Which other sheets does it depend on?
- What do the diagrams/images represent?
- Where does each fact originate?

Avoid merely reproducing cell contents.

---

## 18. Knowledge Chunking

Do not chunk solely by token count.

Prefer semantic chunks such as:

- sheet overview
- business concept
- metric definition
- business rule
- formula explanation
- workflow
- table description
- image interpretation

- cross-sheet relationship
- glossary entry

Every chunk must include source metadata.

Example:

```
{ "chunk_type": "business_rule", "content": "...", "metadata": { "workbook": "sales.xlsx", "sheet": "営業フ
□ー", "range": "B32:F35" } }
```

---

## 19. Embeddings

Embedding generation must operate on semantic knowledge chunks.

Do not embed raw cell-by-cell content unless there is a specific reason.

Each embedding record should contain:

- chunk ID
- text
- chunk type
- workbook
- sheet
- source range
- image ID if applicable

The embedding layer must be replaceable.

Do not tightly couple the application to one embedding provider.

---

## 20. LLM Provider Abstraction

Do not hard-code one LLM.

Create an abstraction supporting providers such as:

- Ollama/local models
- OpenAI-compatible APIs
- other future providers

Configuration should determine the provider.

Example:

```
llm: provider: ollama model: gemma4:12b base_url: http://localhost:11434
```

The code should not contain provider-specific logic throughout the project.

---

## 21. Vision Model Abstraction

Create a separate interface for vision analysis.

The application should support:

- vision-capable local model
- external vision model
- OCR-only fallback

Do not assume every LLM supports images.

---

## 22. Prompt Management

Prompts must live outside Python source code.

Store them under:

prompts/

Use separate prompts for:

- translation
- structure analysis
- image analysis
- formula interpretation
- business rule extraction
- documentation generation

Prompts should request structured JSON whenever machine-readable output is required.

---

## 23. Validation

Every pipeline stage must have validation.

Validate:

- workbook loaded successfully
- sheet count
- image count
- formula count
- IR schema
- required source metadata
- generated documentation
- JSON schema

- no malformed knowledge chunks

Never silently swallow parsing errors.

Errors should be logged with:

- workbook
  - sheet
  - cell/range
  - operation
  - exception
- 

## 24. Idempotency

Running the pipeline twice on the same workbook should produce the same logical result unless the source or configuration changed.

Use deterministic IDs where possible.

Example:

workbook + sheet + source range + element type

can be used to generate stable IDs.

---

## 25. Caching

LLM and vision calls can be expensive.

Implement caching for:

- translations
- image analysis
- formula interpretation
- semantic analysis

A repeated run should not re-call the LLM unnecessarily.

---

## 26. Logging

Use Python logging.

Support:

- DEBUG
- INFO
- WARNING
- ERROR

CLI should expose a verbose/debug option.

Example:

```
python -m excel_knowledge.cli process input.xlsx --verbose
```

---

## 27. CLI

Provide a CLI with commands similar to:

```
excel-knowledge inspect input.xlsx
```

```
excel-knowledge extract input.xlsx
```

```
excel-knowledge analyze input.xlsx
```

```
excel-knowledge document input.xlsx
```

```
excel-knowledge process input.xlsx
```

```
excel-knowledge validate output/
```

The `process` command should run the complete pipeline.

---

## 28. Testing Strategy

Every feature must have tests.

Test:

- simple workbook
- merged cells
- multi-row headers
- formulas
- hidden rows
- hidden columns
- multiple tables
- images
- image anchors

- Japanese text
- mixed Japanese/English
- cross-sheet references
- irregular layouts
- empty sheets
- corrupted/unexpected workbook structures

Use fixtures under:

tests/fixtures/

Do not use real customer-sensitive workbooks in tests.

---

## 29. Error Handling

Never crash the complete workbook pipeline because one cell/image/formula cannot be processed.

Use graceful degradation.

Example:

Image vision analysis failed.

The output should still contain:

- image ID
- source location
- extracted image
- OCR if available
- warning

rather than losing the entire sheet.

---

## 30. Security

Do not log:

- API keys
- passwords
- access tokens
- sensitive workbook contents unnecessarily

Use environment variables for secrets.

Never commit `.env`.

---

## 31. Implementation Order

Implement in this order.

Phase 1: Project setup and configuration.

Phase 2: Workbook inventory and basic extraction.

Phase 3: Intermediate Representation.

Phase 4: Merged cells, styles, hierarchy and table detection.

Phase 5: Image extraction and anchor mapping.

Phase 6: Worksheet rendering and visual analysis.

Phase 7: Japanese-to-English translation.

Phase 8: Formula and business-rule interpretation.

Phase 9: Cross-sheet relationship detection.

Phase 10: English documentation generation.

Phase 11: Semantic chunking.

Phase 12: Embedding abstraction.

Phase 13: Optional Qdrant/RAG integration.

Do not jump directly to RAG.

---

## 32. Coding Rules for Copilot

When implementing a feature:

1. Inspect existing architecture first.
2. Reuse existing models/utilities.
3. Do not duplicate functionality.
4. Add tests with every new feature.
5. Keep functions small and focused.
6. Use type hints.
7. Prefer Pydantic models for structured data.
8. Avoid global mutable state.
9. Do not hard-code paths.
10. Do not hard-code LLM providers.

11. Do not hard-code Japanese translations.
  12. Preserve source traceability.
  13. Preserve original formulas.
  14. Never silently discard images.
  15. Never silently discard parsing failures.
- 

## 33. Definition of Done

A feature is complete only when:

- implementation exists
- unit tests exist
- errors are handled
- logging exists where useful
- source metadata is preserved
- documentation is updated
- configuration is externalized
- the feature works with at least one realistic fixture

The complete project is considered successful when a complex Japanese workbook can be processed into:

1. Workbook inventory
2. Structured Workbook IR
3. Extracted images
4. Image metadata and semantic descriptions
5. English translations
6. Business rules
7. Formula explanations
8. Cross-sheet relationships
9. English Markdown documentation
10. Structured JSON knowledge
11. Semantic knowledge chunks
12. Embeddings
13. Optional RAG retrieval

The system must remain useful even if vision analysis or LLM services are temporarily unavailable by degrading gracefully to deterministic extraction and clearly marking unavailable semantic analysis.